



МИНИСТЕРСТВО ЦИФРОВОГО РАЗВИТИЯ, СВЯЗИ И
МАССОВЫХ КОММУНИКАЦИЙ РОССИЙСКОЙ ФЕДЕРАЦИИ

Ордена Трудового Красного Знамени федеральное государственное
бюджетное образовательное учреждение высшего образования
«**Московский технический университет связи и информатики**»
(МТУСИ)

Кафедра «Системное программирование»

ОТЧЕТ

по лабораторной работе №12
Вариант №26

по дисциплине «**Параллельные и распределенные вычисления**»

Выполнил:

студент гр. БФИ2202

_____ Сидорук Д. В.

« ____ » _____ 2026 г.

Проверил:

старший преподаватель

_____ Клешнин Н. Г.

ассистент

_____ Соловьев А. С.

« ____ » _____ 2026 г.

Москва, 2026 г.

Содержание

1	Введение	3
2	Цель работы	3
3	Задание	3
4	Ход работы	4
	Заключение	10

1 Введение

В данной лабораторной работе рассматривается реализация алгоритма редукции (суммирования) одномерного массива данных на графическом процессоре с использованием технологии CUDA. Актуальность темы обусловлена фундаментальной ролью операции редукции в параллельных вычислениях: она является ключевым этапом во многих алгоритмах машинного обучения, физической симуляции, обработки больших данных и численного анализа, где требуется агрегирование результатов, полученных множеством потоков.

Целью лабораторной работы является изучение принципов организации эффективных параллельных вычислений при выполнении операции суммирования, а также освоение методов оптимизации доступа к памяти GPU. В работе требуется реализовать улучшенное ядро редукции, использующее разделяемую память для минимизации обращений к глобальной памяти и применения паттерна «дерево свёртки». Такой подход позволяет значительно снизить задержки памяти и избежать конфликтов банков, что критически важно для достижения высокой производительности на архитектуре CUDA. Кроме того, предусмотрена обработка списков произвольного размера с корректным учетом граничных условий и финальным подсчетом суммы на центральном процессоре (CPU).

Особенностью рассматриваемой задачи является необходимость балансировки нагрузки между блоками нитей и оптимизация этапов синхронизации внутри блока. Реализация должна обеспечивать обработку входных данных объемом до 2048×65535 элементов за один запуск ядра, используя нейтральные элементы для заполнения незаполненных ячеек разделяемой памяти в последнем блоке. Важным аспектом является сравнение количества операций чтения/записи в глобальную память в базовой и оптимизированной версиях алгоритма, а также анализ влияния структуры дерева редукции на общее количество выполняемых инструкций.

Выполнение лабораторной работы направлено на закрепление навыков разработки высокопроизводительных CUDA-приложений, глубокое понимание иерархии памяти графического ускорителя и механизмов синхронизации потоков. Полученные результаты позволяют оценить эффективность применения стратегий оптимизации памяти и выявить потенциальные узкие места при реализации параллельных алгоритмов сведения данных.

2 Цель работы

Получение навыков выполнения редукции одномерного списка.

3 Задание

Реализовать программу, выполняющую редукцию одномерного списка с помощью CUDA.

4 Ход работы

В листинге ниже приведено содержание файла `rvs_laboratories_11/rvs_laboratories_11.cu`, который содержит код, выполняющий редукцию одномерного списка с помощью CUDA (лист. 1).

Лист. 1 – Содержание `rvs_laboratories_11/rvs_laboratories_11.cu`

```
1 // Имеем список (lst) длины n
2 // Результатом будет его сумма sum = lst[0] + lst[1] + ... +
   ↪ lst[n-1];
3
4 #include "wb.h"
5
6 #define BLOCK_SIZE 512 //@@ Можно поменять
7
8 #define wbCheck(stmt)
   ↪ \
9     do {
   ↪ \
10        cudaError_t err = stmt;
   ↪ \
11        if (err != cudaSuccess) {
   ↪ \
12            wbLog(ERROR, "Failed to run stmt ", #stmt);
   ↪ \
13            return -1;
   ↪ \
14        }
   ↪ \
15    } while (0)
16
17 __global__ void total(float *input, float *output, int len) {
18     __shared__ float sharedData[BLOCK_SIZE];
19
20     unsigned int tID = threadIdx.x;
21     unsigned int gridSize = BLOCK_SIZE << 1;
22     unsigned int idx = blockIdx.x * gridSize + threadIdx.x;
23
24     //@@ Загрузка сегмента входного вектора в разделяемую память
25     float sum = 0.0f;
26     // Загружаем сразу несколько элементов для уменьшения
   ↪ обращений к глобальной памяти
27     if (idx < len) {
28         sum += input[idx];
29     }
30     if (idx + BLOCK_SIZE < len) {
31         sum += input[idx + BLOCK_SIZE];
32     }
33     sharedData[tID] = sum;
34     __syncthreads();
35
36     //@@ Пробег по дереву редукции
```

```

37     for (unsigned int stride = BLOCK_SIZE / 2; stride > 0; stride
    ↪     >>= 1) {
38         if (tID < stride) {
39             sharedData[tID] += sharedData[tID + stride];
40         }
41         __syncthreads();
42     }
43
44     //@@ Запись вычисленной суммы блока в выходной вектор по
    ↪ нужному адресу
45     // Первая нить блока записывает сумму блока в выходной массив
46     if (tID == 0) {
47         output[blockIdx.x] = sharedData[0];
48     }
49 }
50
51 int main(int argc, char **argv) {
52     wbArg_t args;
53     float *hostInput;    // Входной одномерный список
54     float *hostOutput;   // Выходной список
55     float *deviceInput;
56     float *deviceOutput;
57     int numInputElements; // количество элементов входного списка
58     int numOutputElements; // количество элементов выходного списка
59
60     args = wbArg_read(argc, argv);
61
62     wbTime_start(Generic, "Importing data and creating memory on
    ↪     host");
63     hostInput =
64         (float *)wbImport(wbArg_getInputFile(args, 0),
    ↪         &numInputElements);
65
66     numOutputElements = numInputElements / (BLOCK_SIZE << 1);
67     if (numInputElements % (BLOCK_SIZE << 1)) {
68         numOutputElements++;
69     }
70     hostOutput = (float *)malloc(numOutputElements * sizeof(float));
71
72     wbTime_stop(Generic, "Importing data and creating memory on
    ↪     host");
73
74     wbLog	TRACE, "The number of input elements in the input is ",
75             numInputElements);
76     wbLog	TRACE, "The number of output elements in the input is ",
77             numOutputElements);
78
79     wbTime_start(GPU, "Allocating GPU memory.");
80     //@@ Выделение памяти GPU
81     wbCheck(cudaMalloc((void**)&deviceInput, numInputElements *
    ↪     sizeof(float)));
82     wbCheck(cudaMalloc((void**)&deviceOutput, numOutputElements *
    ↪     sizeof(float)));
83     wbTime_stop(GPU, "Allocating GPU memory.");

```

```

84
85 wbTime_start(GPU, "Copying input memory to the GPU.");
86 //@@ Копирование памяти на GPU
87 wbCheck(cudaMemcpy(deviceInput, hostInput, numInputElements *
88     ↪ sizeof(float),
89         cudaMemcpyHostToDevice));
90
91 wbTime_stop(GPU, "Copying input memory to the GPU.");
92
93 //@@ Инициализация размерности блока и сетки
94
95 dim3 blockDim(BLOCK_SIZE);
96 dim3 gridDim(numOutputElements);
97
98 wbTime_start(Compute, "Performing CUDA computation");
99 //@@ Запуск ядра
100 total<<<gridDim, blockDim>>>(deviceInput, deviceOutput,
101     ↪ numInputElements);
102 cudaDeviceSynchronize();
103 wbTime_stop(Compute, "Performing CUDA computation");
104
105 wbTime_start(Copy, "Copying output memory to the CPU");
106 //@@ Копирование памяти обратно с GPU на CPU
107 wbCheck(cudaMemcpy(hostOutput, deviceOutput, numOutputElements *
108     ↪ sizeof(float),
109         cudaMemcpyDeviceToHost));
110 wbTime_stop(Copy, "Copying output memory to the CPU");
111
112 /*****
113     ↪ *****
114     * Выполните редукцию выходного вектора на хосте
115     * ЗАМЕТКА: Операция может быть рекурсивной и поддерживать
116     ↪ выходной
117     * вектор любого размера. Для простоты мы не требуем этого в
118     ↪ этой
119     * лабораторной.
120     *****/
121 // Суммируем частичные суммы на CPU
122 float finalSum = 0.0f;
123 for (int i = 0; i < numOutputElements; i++) {
124     finalSum += hostOutput[i];
125 }
126 hostOutput[0] = finalSum;
127
128 wbTime_start(GPU, "Freeing GPU Memory");
129 //@@ Освободите память GPU
130 wbCheck(cudaFree(deviceInput));
131 wbCheck(cudaFree(deviceOutput));
132 wbTime_stop(GPU, "Freeing GPU Memory");
133
134 wbSolution(args, hostOutput, 1);
135
136 free(hostInput);
137 free(hostOutput);
138
139

```

```

131     return 0;
132 }

```

В листинге ниже приведено содержание файла `dataset_generator/dataset_generator.cpp`, который содержит код, генерирующий тестовые входные и контрольные выходные данные (лист. 2).

Лист. 2 – Содержание `dataset_generator/dataset_generator.cpp`

```

1  #include "gputk.h"
2
3  static char *base_dir;
4
5  static float compute(float *input, int num) {
6      float ret = 0;
7      for (int ii = 0; ii < num; ++ii) {
8          ret += input[ii];
9      }
10     return ret;
11 }
12
13 static float *generate_data(int n) {
14     float *data = (float *)malloc(sizeof(float) * n);
15     for (int i = 0; i < n; i++) {
16         data[i] = ((float)(rand() % 20) - 5) / 5.0f;
17     }
18     return data;
19 }
20
21 static void write_data(char *file_name, float *data, int num) {
22     FILE *handle = fopen(file_name, "w");
23     fprintf(handle, "%d", num);
24     for (int ii = 0; ii < num; ii++) {
25         fprintf(handle, "\n%.2f", *data++);
26     }
27     fflush(handle);
28     fclose(handle);
29 }
30
31 static void create_dataset(int datasetNum, int dim) {
32     const char *dir_name =
33         gpuTKDirectory_create(gpuTKPath_join(base_dir, datasetNum));
34
35     char *input_file_name = gpuTKPath_join(dir_name, "input.raw");
36     char *output_file_name = gpuTKPath_join(dir_name, "output.raw");
37
38     float *input_data = generate_data(dim);
39     float *output_data;
40
41     output_data = compute(input_data, dim);
42
43     write_data(input_file_name, input_data, dim);
44     write_data(output_file_name, &output_data, 1);

```

```

45     free(input_data);
46 }
47
48
49 int main() {
50     base_dir = gpuTKPath_join(gpuTKDirectory_current(), "Reduction",
51                               ↪ "Dataset");
52     create_dataset(0, 16);
53     create_dataset(1, 64);
54     create_dataset(2, 93);
55     create_dataset(3, 112);
56     create_dataset(4, 1120);
57     create_dataset(5, 1233);
58     create_dataset(6, 1033);
59     create_dataset(7, 4098);
60     create_dataset(8, 4018);
61     create_dataset(9, 1);
62     return 0;
63 }

```

В листинге ниже приведено содержание файла CMakeLists.txt, определяющего правила сборки программы (лист. 3).

Лист. 3 – Содержание CMakeLists.txt

```

1  cmake_minimum_required(VERSION 3.27)
2  project(rvs_laboratories_11 LANGUAGES CXX CUDA)
3
4  set(LIBGPU TK_PATH "/usr/local/libgputk" CACHE STRING "Path to
5  ↪ libgputk")
6
7  add_executable(dataset_generator
8  ↪ dataset_generator/dataset_generator.cpp)
9
10 target_include_directories(dataset_generator PRIVATE
11 ↪ ${LIBGPU TK_PATH}/include)
12 target_link_directories(dataset_generator PRIVATE
13 ↪ ${LIBGPU TK_PATH}/lib/)
14 target_link_libraries(dataset_generator gputk)
15
16 add_executable(rvs_laboratories_11
17 ↪ rvs_laboratories_11/rvs_laboratories_11.cu)

```

Сгенерируем тестовые входные данные и проверим, что они действительно были сгенерированы (рис. 1).


```
build: bash — Konsole
eoanermine@eoanermine:~/Repositories/rvs_laboratories/rvs_laboratories_11/build$ ./dataset_generator
eoanermine@eoanermine:~/Repositories/rvs_laboratories/rvs_laboratories_11/build$ ls ./Reduction/Dataset/
0 1 2 3 4 5 6 7 8 9
eoanermine@eoanermine:~/Repositories/rvs_laboratories/rvs_laboratories_11/build$ ls ./Reduction/Dataset/0/
input.raw output.raw
eoanermine@eoanermine:~/Repositories/rvs_laboratories/rvs_laboratories_11/build$ cat ./Reduction/Dataset/0/input.raw
16
-0.40
0.20
2.40
2.00
1.60
2.00
0.20
1.40
0.80
-0.80
-0.60
0.40
1.00
2.80
-0.40
eoanermine@eoanermine:~/Repositories/rvs_laboratories/rvs_laboratories_11/build$ cat ./Reduction/Dataset/0/output.raw
1
12.80eoanermine@eoanermine:~/Repositories/rvs_laboratories/rvs_laboratories_11/build$
```

Рис. 1 – Генерация тестовых входных данных

2) На рисунке ниже приведен результат сборки и выполнения программы (лист.

```
-: rvs_laboratorie — Konsole
eoanermine@eoanermine:~/Repositories/rvs_laboratories/rvs_laboratories_11/build$ cmake ..
-- Configuring done (0.0s)
-- Generating done (0.0s)
-- Build files have been written to: /home/eoanermine/Repositories/rvs_laboratories/rvs_laboratories_11/build
eoanermine@eoanermine:~/Repositories/rvs_laboratories/rvs_laboratories_11/build$ make
-- Configuring done (0.0s)
-- Generating done (0.0s)
-- Build files have been written to: /home/eoanermine/Repositories/rvs_laboratories/rvs_laboratories_11/build
[ 50%] Built target dataset_generator
[100%] Built target rvs_laboratories_11
eoanermine@eoanermine:~/Repositories/rvs_laboratories/rvs_laboratories_11/build$ ./rvs_laboratories_11 ./Reduction/D
ataset/6/input.raw ./Reduction/Dataset/6/output.raw
[Generic] 0.000373248 Importing data and creating memory on host
Trace main::74 The number of input elements in the input is 1033
Trace main::76 The number of output elements in the input is 2
[GPU ] 0.000079872 Allocating GPU memory.
[GPU ] 0.000016896 Copying input memory to the GPU.
[Compute] 0.000114944 Performing CUDA computation
[Copy ] 0.000019968 Copying output memory to the CPU
[GPU ] 0.000056064 Freeing GPU Memory
Solution is correct.
eoanermine@eoanermine:~/Repositories/rvs_laboratories/rvs_laboratories_11/build$
```

Рис. 2 – Компиляция и выполнение программы

Как можно видеть, вычисление редукции было выполнено корректно и успешно.

Заключение

Таким образом, в ходе выполнения данной лабораторной работы были успешно получены навыки выполнения редукции одномерного списка. На практике была реализована программа, выполняющая редукцию одномерного списка с помощью CUDA.